Subword Tokenization: BPE and Unigram Language Model

Diego Besada Rodríguez

[My code] [Pierre Nugues' Template]

September 2026

1 Objectives

Tokenization is the first step in most NLP pipelines: it determines the basic units (tokens) that a model sees. Choosing a good vocabulary is a hard trade-off: word-level tokenizers cannot handle unseen words, while character-level tokenizers produce very long sequences. Subword methods such as Byte Pair Encoding (BPE) and the Unigram language model address this by building a vocabulary of variable-size symbols that interpolate between characters and words.

The experiments use the `herrgard.txt` novel from the Selma Lagerlöf corpus and a target vocabulary of $k = 116$ tokens. The output of each algorithm is compared against the reference implementation in the `sentencepiece` library.

2 Method and Program Structure

Whitespace is replaced by the Unicode character ▁ (U+2581) so that word boundaries remain explicit after tokenization, following `sentencepiece`'s convention. The corpus is then split into a list of individual characters, which serves as the starting point for both algorithms.

2.1 BPE

Byte Pair Encoding (BPE) is a subword tokenization algorithm that starts by splitting the entire corpus into individual characters and then repeatedly merges the most frequent pair of adjacent symbols into a new token. This process continues until the vocabulary reaches the desired size. Because frequent character pairs are merged early, common words like "the" or "and" end up as single tokens, while rare or unseen words get broken down into smaller subword units or even individual characters. This gives a good balance: the vocabulary stays compact, and the model can still represent any word without needing an entry for every possible word in the language. The downside is that BPE is greedy and deterministic. It always picks the most frequent pair at each step, which means it can make suboptimal choices early on that affect all later merges. It also tends to absorb common suffixes and punctuation into adjacent tokens, producing subword units that don't correspond to meaningful linguistic parts. Finally, because it always produces a single unique segmentation, it cannot express alternative ways to split a word.

Our implementation follows the same procedure: we **initialize the vocabulary with all unique characters**, then for k iterations we **count all adjacent pairs** (ignoring those that cross word boundaries), **merge the most frequent pair** into a new token, and **rewrite the corpus** with the merged result. After k merges, tokenizing new text simply means **applying the recorded merge operations in the same order** they were learned.

2.2 Unigram

The Unigram language model takes the opposite approach: instead of building the vocabulary bottom-up by merging, it starts with a large candidate vocabulary and prunes it down to the desired size. It assumes that each subword occurs independently and assigns a probability to every possible segmentation of a sentence. The best segmentation is the one with the highest overall probability, found using the Viterbi algorithm. Token probabilities are estimated from their frequencies using the EM algorithm. For vocabulary construction, the algorithm starts with a seed vocabulary containing all individual characters and the most frequent substrings in the corpus. It then iteratively fits the unigram model, measures how much the overall likelihood would drop if each subword were removed (its "loss"), and discards the subwords with the lowest loss. This top-down pruning avoids the greedy pitfalls of BPE: it produces subword units that align better with actual morphological boundaries and avoids generating rare intermediate tokens.

Our implementation has two phases. For segmentation, we use a **recursive cached search** that tries every possible split point, scores each candidate by the **sum of its token log-probabilities**, and keeps the highest-scoring segmentation. For vocabulary selection, we start from the BPE vocabulary as seed, then iteratively **remove the token whose absence causes the smallest drop in corpus likelihood**, **re-estimate probabilities** via EM, and **repeat** until the vocabulary reaches the target size.

3 Results and Conclusion

After 50 BPE merges, the vocabulary grows from the initial character set to 116 tokens. The resulting tokenization exactly matches the output of `sentencepiece`'s BPE mode, confirmed by an assertion over the full corpus.

For the Unigram model, the vocabulary selection loop converges after a few iterations: the corpus log-likelihood stabilizes at approximately -494583.24. The pruning step measures, for each candidate subword, how much the log-likelihood drops when that subword is temporarily removed. The subword `_o` shows the smallest immediate cost (78.02), so it is selected as the best candidate for removal. After discarding it and re-estimating the probabilities via the EM loop, the log-likelihood improves from -494583.24 to -494534.09, confirming that `_o` was indeed redundant and that removing it allows the model to redistribute probability mass more efficiently.

Both BPE and the Unigram model produce compact, lossless subword vocabularies that generalize beyond whole words. BPE is deterministic and fast—each merge is a single linear scan—but its greedy heuristic can be suboptimal for rare words, as it may allocate vocabulary slots to frequent substructures while leaving unusual ones over-segmented. The Unigram model, by optimizing a global probability objective and evaluating all possible segmentations, is more principled but computationally heavier due to the recursive search. A key difference is that BPE builds its vocabulary bottom-up by frequency, while the Unigram method prunes a large seed vocabulary top-down by measuring each subword's contribution to the overall likelihood. In our experiment, this pruning correctly identified `_o` as the most redundant token: its removal initially appeared to cost 78.02 in log-likelihood, but after re-estimating the token probabilities the model actually improved by ~ 49, demonstrating that the EM loop can compensate and find a better distribution over a smaller vocabulary. In practice, both approaches yield comparable subword decompositions, with the Unigram method providing a cleaner probabilistic framework for downstream tasks.

Relation to GPT-2

The paper *Language Models are Unsupervised Multitask Learners* [3] describes GPT-2, a 1.5B-parameter Transformer trained on WebText, a 45-million-link web scrape filtered by Reddit karma. Its Section 2.2 on input representation is directly relevant to the tokenization work carried out in the lab. GPT-2 uses a byte-level BPE variant that operates on UTF-8 bytes rather than Unicode code points, requiring only a base vocabulary of 256 symbols. To avoid suboptimal merges (e.g., allocating multiple slots to variants of "dog" like `dog.`, `dog!`, `dog?`), the authors prevent BPE from merging across character categories—grouping bytes into classes such as letters, digits, and punctuation, and forbidding any merge that would combine bytes from different categories. They add an exception for spaces, which significantly improves compression efficiency while adding only minimal fragmentation of words across multiple vocab tokens. This mirrors the design choice in our BPE implementation where merges are forbidden across word boundaries (`_`), ensuring that the limited vocabulary budget is spent on meaningful subword units rather than on punctuation variants of common tokens.

More broadly, the GPT-2 paper argues that a sufficiently large language model, trained as a pure next-token predictor on diverse internet text, begins to perform downstream tasks—translation, summarization, question answering—in a zero-shot setting without explicit supervision. This connects to the n-gram and segmentation work from previous labs: just as the bigram model conditions on the previous word to recover word boundaries and the unigram model scores segmentations by token probability, GPT-2 conditions on up to 1024 preceding tokens to predict the next one. The tokenization experiments show that the choice of subword units directly shapes what the model sees at each step; GPT-2's byte-level BPE ensures that every Unicode string is representable, eliminating the out-of-vocabulary problem that plagued word-level models and enabling the zero-shot generalization the paper demonstrates across heterogeneous datasets.

References

[1] Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of ACL*, 2018.

[2] Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Proceedings of EMNLP Findings*, 2020.

[3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.